

Compilador del Lenguaje HULK

Diseño, Implementación y Extensiones

Informe de Proyecto

Asignaturas: *Lenguajes de Programación y Compilación*

Autores:

Darío Francisco Alfonso Urrutia @D4R102004

Juan Carlos Carmenate Díaz @Juank404

Sebastian González Alfonso @sebagonz106

Facultad de Matemática y Computación

Universidad de La Habana

Junio de 2026

Repositorio: <https://github.com/D4R102004/hulk-compiler>

Resumen / Abstract

Resumen

Este informe describe el diseño e implementación de un compilador completo para el lenguaje HULK (*Havana University Language for Kompilers*), desarrollado como proyecto final de las asignaturas Lenguajes de Programación y Compilación de la Universidad de La Habana. El compilador está implementado íntegramente en Rust e incluye un analizador léxico, un analizador sintáctico descendente recursivo LL(1), un analizador semántico de cinco pasadas con inferencia de tipos, y un generador de código basado en LLVM 17 mediante la biblioteca `inkwell`. El compilador cubre todos los aspectos del lenguaje HULK hasta la verificación de tipos (sección 16.8 de la documentación oficial) e introduce tres extensiones al lenguaje: *expresiones **match*** para coincidencia de patrones estructurada, *tipos función* que permiten tratar funciones y métodos como valores de primera clase, y *vectores* con soporte para comprensiones al estilo funcional. El informe analiza las decisiones de diseño más relevantes, las compara con soluciones adoptadas en lenguajes como Kotlin, Rust, Haskell y Python, y documenta las pruebas de validación realizadas.

Abstract

This report describes the design and implementation of a complete compiler for the HULK language (*Havana University Language for Kompilers*), developed as the final project for the Programming Languages and Compilation courses at the University of Havana. The compiler is written entirely in Rust and comprises a hand-written lexer, a hand-written LL(1) recursive-descent parser, a five-pass semantic analyzer with type inference, and a code generator targeting LLVM 17 via the `inkwell` crate. The compiler covers all HULK features through type verification (§16.8) and introduces three language extensions: **match expressions** for structured pattern matching, **function types** enabling first-class function values, and **vectors** with functional-style comprehension syntax. The report analyzes the key design decisions, compares them to solutions in Kotlin, Rust, Haskell, and Python, and documents the validation test suite.

Palabras clave: compiladores, HULK, Rust, LLVM, inferencia de tipos, coincidencia de patrones, funciones de primera clase, vectores.

Índice

1. Introducción	3
1.1. Historia de la compilación y los lenguajes de programación	3
1.2. El lenguaje HULK y el contexto del proyecto	3
1.3. Motivación y objetivos de las extensiones	4
1.4. Estructura del informe	4
2. Descripción general del lenguaje HULK	5
2.1. Paradigma y filosofía del lenguaje	5
2.2. Sistema de tipos	5
2.2.1. Tipos primitivos y jerarquía	5
2.2.2. Inferencia de tipos	6
2.3. Características principales del lenguaje	6
2.3.1. Variables y ámbito léxico	6
2.3.2. Funciones	7
2.3.3. Orientación a objetos	7
2.3.4. Protocolos	8
2.3.5. Control de flujo	8
2.4. El analizador léxico	9
3. Extensiones propuestas	10
3.1. Extensión 1: Expresiones <code>match</code>	10
3.1.1. Nombre y propósito	10
3.1.2. Sintaxis	10
3.1.3. Semántica estática	11
3.1.4. Semántica dinámica	11
3.1.5. Comparativa con otros lenguajes	11
3.2. Extensión 2: Tipos función y funciones de primera clase	12
3.2.1. Nombre y propósito	12
3.2.2. Sintaxis	12
3.2.3. Semántica estática	13
3.2.4. Semántica dinámica	13
3.2.5. Comparativa con otros lenguajes	13
3.3. Extensión 3: Vectores con comprensiones	14
3.3.1. Nombre y propósito	14
3.3.2. Sintaxis	14
3.3.3. Semántica estática	15
3.3.4. Comparativa con otros lenguajes	15

1 Introducción

1.1 Historia de la compilación y los lenguajes de programación

El nacimiento de los compiladores está íntimamente ligado al origen de la programación moderna. En 1952, Grace Hopper desarrolló el primer compilador conocido, el A-0 System, que traducía código simbólico matemático a instrucciones de máquina. Este hito marcó el inicio de una nueva era: por primera vez, los programadores podían expresar algoritmos en un nivel de abstracción superior al del lenguaje ensamblador.

El salto definitivo llegó en 1957 con FORTRAN (*Formula Translation*), desarrollado por John Backus y su equipo en IBM. FORTRAN demostró que era posible generar código máquina tan eficiente como el escrito a mano, disipando las dudas de quienes creían que la abstracción debía sacrificar rendimiento. A FORTRAN le siguió LISP en 1958, introduciendo la recursión y el tratamiento de funciones como objetos de primera clase —conceptos que siguen siendo relevantes en el diseño de lenguajes actuales.

Las décadas de 1960 y 1970 vieron la consolidación de la teoría formal detrás de la compilación. Noam Chomsky formuló la jerarquía de gramáticas formales, mientras que Donald Knuth definió los lenguajes LR(k), sentando las bases matemáticas del análisis sintáctico. La publicación del “libro del dragón” (*Compilers: Principles, Techniques, and Tools*) por Aho, Sethi y Ullman en 1986 codificó el conocimiento acumulado en una referencia canónica que aún se utiliza en cursos universitarios de todo el mundo.

El estado del arte moderno incluye compiladores de infraestructura compleja como GCC y LLVM. Este último, desarrollado originalmente por Chris Lattner en 2003, introdujo un concepto transformador: una *representación intermedia* (IR) de bajo nivel y tipada que permite separar completamente el *front end* (análisis y tipado) del *back end* (generación de código nativo). Hoy en día, LLVM es la columna vertebral de compiladores para lenguajes como Clang (C/C++), Swift, Kotlin Native y Rust.

1.2 El lenguaje HULK y el contexto del proyecto

HULK (*Havana University Language for Kompilers*) es un lenguaje de programación didáctico diseñado en la Facultad de Matemática y Computación de la Universidad de La Habana, concebido específicamente para el estudio de la compilación. HULK es un lenguaje de tipado estático, orientado a objetos y basado en expresiones: toda construcción sintáctica —incluyendo bloques, condicionales y bucles— produce un valor.

El lenguaje incluye un sistema de tipos con inferencia, herencia simple, polimorfismo, protocolos (equivalentes a interfaces estructurales), iterables y vectores. Su diseño deliberadamente compacto permite a un equipo de estudiantes implementar un compilador completo en el transcurso de un semestre, cubriendo todas las fases clásicas: análisis léxico, sintáctico, semántico y generación de código.

El presente proyecto, desarrollado como trabajo final de las asignaturas Lenguajes de Programación y Compilación, consiste en la implementación de un compilador completo para HULK en el lenguaje Rust, generando código nativo para Linux x86_64 mediante el *backend* LLVM 17.

1.3 Motivación y objetivos de las extensiones

La especificación base de HULK cubre los mecanismos fundamentales de la programación orientada a objetos con tipos estáticos. Sin embargo, el lenguaje original carece de herramientas expresivas modernas que permiten escribir código más conciso, seguro y composable. Este proyecto introduce tres extensiones al lenguaje, cada una motivada por necesidades concretas:

1. **Expresiones `match`.** La ausencia de coincidencia de patrones obliga a escribir cadenas de `if/elif/else` verbosas y propensas a errores. La expresión `match` permite inspeccionar el valor y el tipo de una expresión de forma declarativa, siguiendo el modelo de lenguajes como Rust, Kotlin y Haskell.
2. **Tipos función y funciones de primera clase.** En HULK base, las funciones son entidades globales y no pueden pasarse como argumentos ni almacenarse en variables. Esta extensión introduce el tipo $(T_1, \dots, T_n) \rightarrow R$ para representar funciones como valores, habilitando la programación funcional de alto orden: paso de funciones como argumentos, devolución de funciones desde funciones y referencias a métodos.
3. **Vectores con comprensiones.** Las colecciones con tipo homogéneo son esenciales en cualquier lenguaje práctico. Esta extensión añade el tipo `Vector<T>` con sintaxis de literal `[1, 2, 3]` e indexación, así como comprensiones de vectores al estilo funcional `[expr | var in iterable]`.

Los tres objetivos de diseño que guían estas extensiones son: (1) *consistencia* con la semántica basada en expresiones de HULK, (2) *seguridad de tipos* verificada estáticamente, y (3) *expresividad* comparable a la de lenguajes modernos como Kotlin y Rust.

1.4 Estructura del informe

El resto del informe se organiza de la siguiente manera. La Sección 2 describe el lenguaje HULK original, sus paradigmas y su sistema de tipos. La Sección 3 presenta en detalle las tres extensiones propuestas, con su sintaxis formal, reglas de tipado, semántica dinámica y comparativas con otros lenguajes. La Sección ?? analiza la arquitectura completa del compilador y justifica las decisiones de diseño más relevantes. La Sección ?? documenta la metodología de pruebas y los casos más significativos. La Sección ?? discute las dificultades encontradas y las alternativas descartadas. Finalmente, la Sección ?? presenta las conclusiones y el trabajo futuro.

2 Descripción general del lenguaje HULK

2.1 Paradigma y filosofía del lenguaje

HULK (*Havana University Language for Kompilers*) es un lenguaje de programación estáticamente tipado, orientado a objetos y **basado en expresiones**. A diferencia de lenguajes como C o Java, donde las construcciones de control de flujo son *sentencias* (sin valor de retorno), en HULK toda construcción produce un valor: los bloques, los condicionales, los bucles y la coincidencia de patrones son expresiones que pueden aparecer en cualquier posición donde se espere un valor. Esta filosofía, compartida con lenguajes como Rust, Scala y Kotlin, simplifica la semántica del lenguaje y fomenta un estilo de programación más declarativo.

El programa HULK más sencillo ilustra esta idea:

```
1 print("Hello, World!");
```

Listing 1: Programa mínimo en HULK

Un programa HULK está compuesto por una secuencia de *declaraciones globales* (funciones, tipos y protocolos) seguida de una única *expresión de entrada* que constituye el punto de inicio de la ejecución.

2.2 Sistema de tipos

HULK implementa un sistema de tipos estático con inferencia parcial. Los tipos se organizan en una jerarquía nominal con `Object` como raíz, y el sistema admite tanto anotaciones explícitas como inferencia automática en la mayoría de los contextos.

2.2.1 Tipos primitivos y jerarquía

El sistema de tipos del compilador implementado define los siguientes tipos fundamentales, representados como variantes del enum `Type` en el módulo `hulk-semantic`:

Cuadro 1: Tipos primitivos del sistema de tipos de HULK

Tipo HULK	Variante interna	Descripción
<code>Number</code>	<code>Type::Number</code>	Número de punto flotante (f64)
<code>String</code>	<code>Type::String</code>	Cadena de texto inmutable
<code>Boolean</code>	<code>Type::Boolean</code>	Valor lógico (<code>true/false</code>)
<code>Object</code>	<code>Type::Object</code>	Raíz de la jerarquía nominal
<code>T (usuario)</code>	<code>Type::Named(String)</code>	Tipo o protocolo definido por el usuario
<code>Vector<T></code>	<code>Type::Vector(Box<Type>)</code>	Colección homogénea (extensión)
<code>T*</code>	<code>Type::Iterable(Box<Type>)</code>	Protocolo iterable
<code>(T)->R</code>	<code>Type::Function{...}</code>	Tipo función (extensión)

Los tipos `Number`, `String` y `Boolean` son tipos de valor primitivos: no pueden ser heredados y se representan sin boxing en la generación de código. `Object` es el supertipo universal — todo tipo HULK conforma a `Object`. Los tipos `Vector<T>` y `(T)->R` son extensiones introducidas en este proyecto, descritas en detalle en la Sección 3.

La relación de conformidad (*subtyping*) $T_1 \leq T_2$ se implementa en el método `conforms_to` y sigue las reglas:

1. **Reflexividad:** $T \leq T$ para todo T .
2. **Top:** $T \leq \text{Object}$ para todo T .
3. **Herencia nominal:** $T \leq U$ si U es ancestro de T en la jerarquía de herencia.
4. **Conformidad estructural a protocolos:** $T \leq P$ (protocolo) si T implementa todos los métodos de P con las signaturas correctas.
5. **Covarianza de Vector:** $\text{Vector} < T > \leq \text{Vector} < U >$ si $T \leq U$.
6. **Tipos función:** $(A_1, \dots, A_n) \rightarrow R_1 \leq (B_1, \dots, B_n) \rightarrow R_2$ si $B_i \leq A_i$ (contravarianza en parámetros) y $R_1 \leq R_2$ (covarianza en retorno).

2.2.2 Inferencia de tipos

HULK permite declarar funciones sin anotaciones de tipo explícitas. El compilador infiere los tipos a partir del uso:

```

1 // Sin anotaciones: el compilador infiere Number -> Number
2 function square(x) {
3     x * x;
4 }
5
6 // Con anotaciones explícitas
7 function add(x: Number, y: Number): Number {
8     x + y;
9 }

```

Listing 2: Inferencia de tipos en HULK

El análisis semántico implementado resuelve la inferencia de tipos en cinco pasadas ordenadas (ver Sección ??).

2.3 Características principales del lenguaje

2.3.1 Variables y ámbito léxico

HULK usa `let ... in` como construcción de enlace de variables. El ámbito es léxico y las variables pueden ser shadowed en ámbitos internos:

```

1 let x = 10 in {
2     if (x == 10) print("ok") else print("fail");
3     let x = 20 in {           // x queda en la sombra (shadowing)
4         if (x == 20) print("ok") else print("fail");
5     };
6     if (x == 10) print("ok") else print("fail");
7 };

```

Listing 3: Variables y shadowing en HULK

La asignación destructiva `:=` modifica el valor de una variable existente sin crear un nuevo enlace:

```

1 let i = 0 in
2 let result = 0 in {
3   while (i < 5) {
4     result := result + i;
5     i := i + 1;
6   };
7   print(result); // imprime 10
8 };

```

Listing 4: Asignación destructiva

2.3.2 Funciones

Las funciones en HULK son globales y admiten dos formas sintácticas: la forma *inline* con `=>` y la forma de bloque con `{}`. Soportan recursión y referencias mutuas (gracias al paso de recolección de declaraciones que registra todas las signaturas antes de inferir los cuerpos):

```

1 function is_even(n: Number): Boolean {
2   if (n == 0) true else is_odd(n - 1);
3 }
4
5 function is_odd(n: Number): Boolean {
6   if (n == 0) false else is_even(n - 1);
7 }

```

Listing 5: Recursión mutua en HULK

Las funciones builtin disponibles son: `print`, `sqrt`, `sin`, `cos`, `exp`, `log`, `rand` y `range`. Además, `PI` y `E` se modelan como funciones de aridad cero que actúan como constantes matemáticas.

2.3.3 Orientación a objetos

HULK implementa orientación a objetos con herencia simple y polimorfismo. Los tipos se declaran con la palabra clave `type`, pueden recibir parámetros de constructor e implementar métodos. Todos los atributos son privados (accesibles solo dentro del tipo que los declara) y todos los métodos son públicos y virtuales:

```

1 type Animal(n: String) {
2   name: String = n;
3   sound(): String { "..."; }
4 }
5
6 type Dog(n: String) inherits Animal(n) {
7   sound(): String { "Woof"; }
8 }
9
10 type Cat(n: String) inherits Animal(n) {
11   sound(): String { "Meow"; }

```

```

12 }
13
14 // Polimorfismo: variable de tipo Animal contiene un Dog
15 let a: Animal = new Dog("Rex") in print(a.sound()); // "Woof"

```

Listing 6: Tipos, herencia y polimorfismo en HULK

La palabra clave `base` permite delegar al método del padre inmediato:

```

1 type FancyPrinter(prefix: String, suffix: String)
2     inherits Printer(prefix) {
3     sfx: String = suffix;
4     format(msg: String): String {
5         base(msg) @ self.sfx; // delega a Printer.format y agrega
6         sufijo
7     }
8 }

```

Listing 7: Uso de `base` para delegación

2.3.4 Protocolos

Los protocolos en HULK definen interfaces estructurales: un tipo implementa un protocolo si posee todos sus métodos con las firmas correctas, sin necesidad de declararlo explícitamente. Esto es *tipado estructural* (similar a los *traits* de Go o las *interfaces implícitas* de TypeScript):

```

1 protocol Printable {
2     to_string(): String;
3 }
4
5 type Point(x: Number, y: Number) {
6     x: Number = x;
7     y: Number = y;
8     to_string(): String { "point"; }
9     // Point implementa Printable impl citamente
10 }
11
12 let p: Printable = new Point(1, 2) in
13     print(p.to_string());

```

Listing 8: Protocolos como interfaces estructurales

2.3.5 Control de flujo

HULK ofrece las construcciones de control clásicas, todas como expresiones: `if/elif/else`, `while` y `for`. El bucle `for` itera sobre cualquier expresión de tipo `Iterable<T>`, incluyendo rangos y vectores:

```

1 // if/elif/else como expresi n
2 function grade(score: Number): String {
3     if (score >= 90) "A"
4     elif (score >= 80) "B"

```

```

5     elif (score >= 70) "C"
6     else "F";
7 }
8
9 // for sobre un rango
10 for (x in range(1, 6))
11     print(x);

```

Listing 9: Control de flujo en HULK

2.4 El analizador léxico

El lexer de HULK, implementado manualmente en el crate `hulk-lexer`, reconoce los siguientes grupos de tokens:

Cuadro 2: Categorías de tokens del lexer de HULK

Categoría	Tokens
Literales	Number(f64), StringLit(String), True, False
Aritmética	Plus, Minus, Star, Slash, Caret, Percent
Cadenas	At (@), AtAt (@@)
Comparación	EqEq, Neq, Lt, Gt, Leq, Geq
Booleanos	And, Or, Not
Asignación	Assign (=), ColonEq (:=)
Delimitadores	LParen, RParen, LBrace, RBrace, LBracket, RBracket
Separadores	Semicolon, Comma, Colon, Dot
Flechas	Arrow (->), FatArrow (=>)
Palabras clave	let, in, if, elif, else, while, for, function, type, inherits, new, self, base, is, as, protocol, match, case

El lexer detecta y reporta los siguientes errores léxicos: cadenas sin terminar (`UnterminatedString`), caracteres no reconocidos (`UnexpectedChar`) y secuencias de escape inválidas (`InvalidEscape`). Notably, la palabra `base` es un *keyword contextual*: el lexer la emite como `TokenKind::Base`, pero el parser la acepta también como identificador en posiciones de nombre de variable, atributo o parámetro —siguiendo el patrón de palabras clave contextuales de C# y SoulNG.

3 Extensiones propuestas

Este proyecto introduce tres extensiones al lenguaje HULK original, todas implementadas con modificaciones reales a la sintaxis, el AST y el análisis semántico. Cada extensión está motivada por carencias concretas del lenguaje base y se justifica mediante comparativas con lenguajes modernos.

3.1 Extensión 1: Expresiones match

3.1.1 Nombre y propósito

La **expresión match** introduce coincidencia de patrones estructurada en HULK. El lenguaje base solo ofrece `if/elif/else` para discriminar valores, lo que resulta en código verboso cuando se necesita comparar un valor contra múltiples casos. La expresión `match`, siendo una expresión (no una sentencia), produce un valor que puede ser usado directamente en cualquier contexto.

3.1.2 Sintaxis

La gramática extendida para `match` es:

```

1 MatchExpr  ::= 'match' Expr '{' MatchCase+ '}'
2 MatchCase  ::= 'case' Pattern '=>' Expr ';'
3 Pattern    ::= '_'                                -- wildcard
4             | Literal                             -- literal value
5             | Identifier                           -- variable binding
6             | TypeRef (',' Identifier)?           -- type pattern

```

Listing 10: Gramática BNF de match

Ejemplos de cada tipo de patrón:

```

1 let x = 42 in
2   match x {
3     case 1  => print("uno");
4     case 42 => print("cuarenta y dos");
5     case _  => print("otro");
6   };

```

Listing 11: Patrones literales

```

1 type Animal { sound(): String { "..."; } }
2 type Dog inherits Animal { sound(): String { "Woof"; } }
3 type Cat inherits Animal { sound(): String { "Meow"; } }
4
5 let a: Animal = new Dog() in
6   match a {
7     case d: Dog => print(d.sound()); // narrows to Dog
8     case c: Cat => print(c.sound()); // narrows to Cat
9     case _      => print("unknown");
10  };

```

Listing 12: Patrones de tipo con jerarquía de herencia

```

1 function describe(s: String): String {
2     match s {
3         case "hello" => "greeting";
4         case "bye"   => "farewell";
5         case _       => "unknown";
6     }
7 }
8 print(describe("hello")); // greeting

```

Listing 13: match como expresión — el resultado se asigna

3.1.3 Semántica estática

El analizador semántico implementa la inferencia de `match` en la función `infer_match` del módulo `hulk-semantic`. Las reglas de tipado son:

1. El escrutinio (expresión después de `match`) se infiere normalmente.
2. Cada caso infiere su cuerpo en un entorno extendido según el patrón:
 - Wildcard (`_`): no añade bindings.
 - Literal: verifica conformidad de tipos con el escrutinio.
 - Variable(`name`): enlaza `name` al tipo del escrutinio.
 - Type(`T`, `binding`): verifica que `T` sea un tipo conocido; si hay `binding`, lo enlaza con tipo `T` (narrowing).
3. El tipo del `match` completo es el *ancestro común más bajo* (LCA) de los tipos de todos los cuerpos de caso.
4. Si ningún caso es *catch-all* (`_` o variable), el compilador emite una advertencia `NonExhaustiveMatch`.

3.1.4 Semántica dinámica

En tiempo de ejecución, la evaluación de `match x { case p => e }` procede así:

1. Se evalúa `x` una sola vez y se guarda el resultado.
2. Los casos se evalúan en orden de declaración.
3. Para el primer patrón que coincida, se evalúa su cuerpo y se retorna el resultado.
4. Si ningún patrón coincide, el runtime llama a `hulk_rt_match_fail()` que termina el programa.

El matching para patrones de tipo usa `hulk_rt_downcast_check`, que recorre la cadena de `vtables` para verificar si el objeto es una instancia del tipo esperado.

3.1.5 Comparativa con otros lenguajes

La decisión de emitir una *advertencia* (no un error) para matches no exhaustivos sigue el modelo de Kotlin, que también permite matches no exhaustivos pero avisa al programador. Rust, en cambio, los rechaza en compilación — una decisión más segura pero que dificulta el desarrollo incremental.

Cuadro 3: Comparativa de match/switch en diferentes lenguajes

Lenguaje	Construcción	Diferencias con HULK
Rust	<code>match</code>	Exhaustividad obligatoria; patrones destructurantes de structs/enums
Kotlin	<code>when</code>	Puede ser expresión o sentencia; soporte para rangos
Haskell	<code>case of</code>	Exhaustividad verificada; guardas con <code> </code>
Python	<code>match</code> (3.10+)	Patrones estructurales; desestructuración de secuencias
Java	<code>switch</code> (14+)	Expresión switch con <code>-></code> ; sin type narrowing automático
HULK	<code>match</code>	Siempre expresión; type narrowing; advertencia si no-exhaustivo

3.2 Extensión 2: Tipos función y funciones de primera clase

3.2.1 Nombre y propósito

En HULK base, las funciones son entidades globales y no pueden almacenarse en variables ni pasarse como argumentos. Esta extensión introduce el tipo **función** $(T_1, \dots, T_n) \rightarrow R$, que permite tratar funciones y métodos como valores de primera clase. Esto habilita la programación funcional de alto orden: composición de funciones, paso de callbacks y referencias a métodos.

3.2.2 Sintaxis

El tipo función se escribe como:

```
1 TypeRef ::= ... | '(' TypeList? ')' ' ->' TypeRef
```

Listing 14: Sintaxis del tipo función

Ejemplo de uso:

```
1 function apply(f: (Number) -> Number, x: Number): Number {
2     f(x);
3 }
4
5 {
6     if (apply(function (x: Number): Number -> x + 1, 5) == 6)
7         print("ok")
8     else
9         print("fail");
10 }
```

Listing 15: Función de alto orden con tipo función

Las referencias a métodos se obtienen accediendo al método sin llamarlo:

```
1 type Counter(init: Number) {
2     val: Number = init;
```

```

3     inc(): Number { self.val := self.val + 1; self.val; }
4 }
5
6 let c = new Counter(0) in {
7     let f: () -> Number = c.inc in { // referencia al m todo
8         print(f()); // llama c.inc()
9         print(f()); // llama c.inc() de nuevo
10    };
11 };

```

Listing 16: Referencia a método como valor

3.2.3 Semántica estática

El sistema de tipos extiende el enum `Type` con la variante:

```

1 pub enum Type {
2     // ... tipos existentes ...
3     /// Tipo funci n: (params) -> return_type.
4     /// Usado para referencias a m todos y expresiones lambda.
5     Function {
6         params: Vec<Type>,
7         return_type: Box<Type>,
8     },
9     // ...
10 }

```

Listing 17: Variante `Type::Function` en Rust

La relación de subtipado para tipos función sigue la regla estándar de la teoría de tipos: **contravarianza en parámetros, covarianza en retorno**:

$$\frac{B_i \leq A_i \quad R_1 \leq R_2}{(A_1, \dots, A_n) \rightarrow R_1 \leq (B_1, \dots, B_n) \rightarrow R_2}$$

Esta regla garantiza que una función más específica puede usarse donde se espera una función más general.

Cuando el calle de una llamada tiene tipo `Type::Function`, el analizador semántico extrae los tipos de parámetros y retorno, verifica la aridad y los tipos de los argumentos, y produce un `CallExpr` tipado con el tipo de retorno como anotación.

3.2.4 Semántica dinámica

Una referencia a método `obj.method` se representa en tiempo de ejecución como un **thunk de dos palabras**: un puntero a la función y un puntero al receptor (`self`). Esto permite llamar al método más tarde sin conocer el tipo concreto del receptor.

3.2.5 Comparativa con otros lenguajes

La decisión de usar un thunk de dos palabras para referencias a métodos sigue el modelo de Kotlin y C#, donde `obj::method` crea un delegate que captura el receptor. En Rust, las referencias a métodos requieren closures explícitas (`|x| obj.method(x)`), lo cual es más verboso.

Cuadro 4: Funciones de primera clase en diferentes lenguajes

Lenguaje	Tipo función	Subtyping
Kotlin	(T) ->R	Contravariante/covariante; función vs. tipo
Rust	fn(T) ->R / Fn(T) ->R	Traits; closures capturan entorno
Haskell	T ->R	Curried; totalmente de primera clase
Java	Function<T,R>	Interfaces funcionales; lambdas desde Java 8
TypeScript	(x: T) =>R	Estructural; compatible por forma
HULK	(T) ->R	Contravariante/covariante; thunk para métodos

3.3 Extensión 3: Vectores con comprensiones

3.3.1 Nombre y propósito

Esta extensión añade el tipo `Vector<T>` como colección homogénea de tamaño fijo con soporte para:

- Literales: `[1, 2, 3]`
- Indexación: `v[0]`
- Métodos: `v.size()`, `v.get(i)`, `v.set(i, x)`
- Comprensiones: `[expr | var in iterable]`
- Iteración con `for`

3.3.2 Sintaxis

```

1 VectorExpr ::= '[' VectorBody
2 VectorBody ::= ']'
3             | ExprNoTopOR VectorTail
4 VectorTail ::= '[' id 'in' Expr ']' -- comprehension
5             | '(' Expr '*' ')'      -- literal
6 TypeRef    ::= TypeRef '['         -- azucar: T[] = Vector<T>
7             | TypeRef '*'          -- azucar: T* = Iterable<T>

```

Listing 18: Gramática BNF de vectores

```

1 let v = [1, 2, 3, 4, 5] in {
2   print(v[0]); // 1
3   print(v.size()); // 5
4 };

```

Listing 19: Literal de vector e indexación

```

1 // Cuadrados de 1 a 5
2 let squares = [x * x | x in range(1, 6)] in
3   for (s in squares)
4     print(s);
5 // Salida: 1 4 9 16 25

```

Listing 20: Comprensión de vector al estilo funcional

Un detalle notable del parser es la resolución de la ambigüedad entre el operador booleano `|` y el separador de comprensión. La gramática usa una producción especial `ExprNoTopOR` para el lado izquierdo de la comprensión, que no consume `|` en el nivel superior. Esto permite que `[(x | y) | x in values]` sea válido porque el OR interno está entre paréntesis.

3.3.3 Semántica estática

La inferencia de vectores implementada en `infer_vector` sigue estas reglas:

1. **Literal:** el tipo del vector es `Vector<T>` donde `T` es el LCA de los tipos de todos los elementos.
2. **Comprensión:** el tipo es `Vector<T>` donde `T` es el tipo de la expresión de elemento, después de inferir el tipo de la variable de iteración desde el iterable.
3. **Indexación:** `v[i]` requiere que `v` sea `Vector<T>` e `i` sea `Number`; retorna `T`.
4. **Covarianza:** `Vector<T> ≤ Vector<U>` si `T ≤ U`.
5. **Iterabilidad:** `Vector<T>` implementa `Iterable<T>` automáticamente.

3.3.4 Comparativa con otros lenguajes

Cuadro 5: Colecciones y comprensiones en diferentes lenguajes

Lenguaje	Colección	Comprensión
Python	<code>list[T]</code>	<code>[x*x for x in range(5)]</code>
Haskell	<code>[T]</code>	<code>[x*x x <- [1..5]]</code>
Kotlin	<code>List<T></code>	<code>(1..5).map { it * it }</code>
Rust	<code>Vec<T></code>	<code>(1..6).map(x x*x).collect()</code>
Java	<code>List<T></code>	Streams: <code>.stream().map(...).collect(...)</code>
HULK	<code>Vector<T></code>	<code>[x*x x in range(1,6)]</code>

La sintaxis de comprensión de HULK (`[expr | var in iter]`) es deliberadamente similar a la de Haskell y Python, que son las más concisas y legibles. La decisión de usar `|` como separador (en lugar de `for` como en Python) sigue el modelo matemático de la notación de conjuntos: $\{x^2 \mid x \in \mathbb{N}\}$.